SituationReport

testdata_generator

Benutzerhandbuch

Demo-Datenräume und synthetische Jira-Daten für SituationReport

---

situation-report -- github.com/Jaegerfeld/situation-report

Für Agile Coaches, PI Manager und Beta-Tester -- Version 0.54.0

1 Was ist der Testdaten-Generator?

Der **Testdaten-Generator** erzeugt erfundene, aber realistische Daten für SituationReport — damit du Portfolio-Report, Delta-Briefing, Konferenzmappe und die Register vorführen, lernen und testen kannst, ohne echte Projektdaten anzufassen. Es kommen keine Personen vor; gleicher Seed ergibt denselben Datenraum.

Der Generator hat zwei Wege. Der **Datenraum** ist der Standardweg der Beta: In drei Schritten entsteht ein kompletter Portfolio-Ordner mit Struktur, Geschichte, Jira-Exporten je ART, Registern, Provider-Rohdaten, einer Reihe von Ständen und den fertigen Delta-Briefings. Der **einzelne ART** ist der klassische Weg: eine Jira-Export-Datei aus einer Workflow-Datei, mit allen Reglern für Cycle Time und Flow-Muster (Kapitel 8).

> **Welcher Weg wann?**
> - **Datenraum (Fenster oder Browser):** Schulung, Präsentation, Tutorials, Tests der Portfolio-Kette — alles, was ein ganzes Lagebild braucht.
> - **Einzelner ART:** ein Jira-Export für transform_data und build_reports, gezielte Flow-Muster für Trainings zu Cycle-Time-Scatterplots, Fortschreibung eines früheren Abzugs.

Dieses Handbuch beschreibt zuerst den Datenraum (Kapitel 2 bis 7) und danach den klassischen Einzel-ART-Generator (Kapitel 8). Wer nur die eine Minute bis zum ersten Portfolio-Report sucht, nimmt das Tutorial „Demo-Datenraum in einer Minute“ (docs/tutorials/demo-data-room.de.md, auch als PDF).

> **Typische Einsatzgebiete:**
> - **Neue Installation testen:** Läuft die Kette von den Rohdaten bis zum Portfolio-Report?
> - **Demos und Präsentationen:** eine Lage mit Ausreißer, schwacher Quelle und Nähten, ohne Datenschutzfrage
> - **Schulungen:** dieselbe Geschichte bei allen Teilnehmenden, reproduzierbar über den Seed
> - **Entwicklung:** Stresstest mit großer Struktur (Umfang L, bis 49 ARTs)

Stand: Beta, Oktober 2026. Die Oberfläche des Datenraums ist neu; der Reiter für einzelne ART-Profile in der Oberfläche fehlt noch (Kommandozeile: --art-profiles).

2 Start

2.1 Portables Paket (empfohlen)

Das portable Paket enthält Python bereits eingebettet — keine Installation. Nach dem Entpacken liegen vier Startdateien für den Generator bereit:

Startdatei (Windows)	Was sie öffnet
`DatenraumFenster.bat`	**Datenraum als Fenster** — der Standardweg. Drei Schritte, kein Netzwerkport, läuft auch auf Firmenrechnern.
`SituationReport.bat`	Der Launcher mit allen Modulen; darin die Karten „Datenraum (Fenster)“ und „Datenraum (Browser)“ sowie „Testdata Generator“ für den einzelnen ART.
`DatenraumBrowser.bat`	**Datenraum im Browser** — dieselben drei Schritte als Webseite. Startet einen lokalen Server auf 127.0.0.1 mit Zufallsschlüssel im Link. Nur auf privaten Rechnern: Sicherheitssoftware auf Firmenrechnern blockiert und meldet lokale Server.
`TestdataGenerator.bat`	**Einzelner ART** — das klassische Fenster mit Workflow-Datei, Reglern und Flow-Mustern (Kapitel 8).

macOS: dieselben Namen als .command (Rechtsklick → Öffnen). Linux: als .sh (python3 und python3-tk vorausgesetzt). Der erste Start richtet einmal eine Python-Umgebung ein — dafür ist Internet nötig.

Hinweis für Windows: Meldet der Start, dass Python nicht gefunden wird, war die ZIP-Datei durch Windows blockiert. Rechtsklick auf die ZIP → Eigenschaften → Sicherheit → Zulassen → OK → erneut entpacken.

2.2 Aus dem Quellcode (Python erforderlich)

Die drei Oberflächen entsprechen drei Aufrufen: Fenster, Browser, einzelner ART.

```
python -m testdata_generator --dataspace
python -m testdata_generator --web
python -m testdata_generator
```

2.3 Fenster oder Browser?

Beide zeigen dieselben drei Schritte mit denselben Einstellungen und erzeugen denselben Datenraum. Das Fenster ist ein normales Programmfenster (tkinter) und braucht nichts weiter. Die Browser-Seite ist etwas bequemer zu lesen, läuft aber über einen lokalen Server — auf einem Firmenrechner hat die Endgeräte-Sicherheit das im ersten Test blockiert und gemeldet. Deshalb gilt: Fenster zuerst, Browser nur privat.

2.4 Sprachen

Oberfläche, Geschichte, README, Fortschritt, Protokoll und die erzeugten Delta-Briefings gibt es in Deutsch, Englisch, Rumänisch, Portugiesisch und Französisch. Der Generator startet in der Sprache, die im Launcher eingestellt ist; oben rechts in Fenster und Browser-Seite lässt sie sich umschalten — Eingaben bleiben erhalten. Ein Lauf hat eine Sprache: Oberfläche und erzeugte Texte sind gleich. Ohne Launcher-Einstellung und ohne `--lang` ist die Sprache Englisch. Rumänisch, Portugiesisch und Französisch sind Entwürfe, die noch von Muttersprachlern geprüft werden.

3 Der Datenraum in drei Schritten

Fenster und Browser-Seite führen durch drei Schritte: **Szenario** (welche Lage), **Umfang & Quellen** (wie groß, wie lang, wie echt) und **Erzeugen** (wohin, und los). Die Standardwerte ergeben die Demo-Lage, die auch Tutorials und Handbücher der Portfolio-Module verwenden.

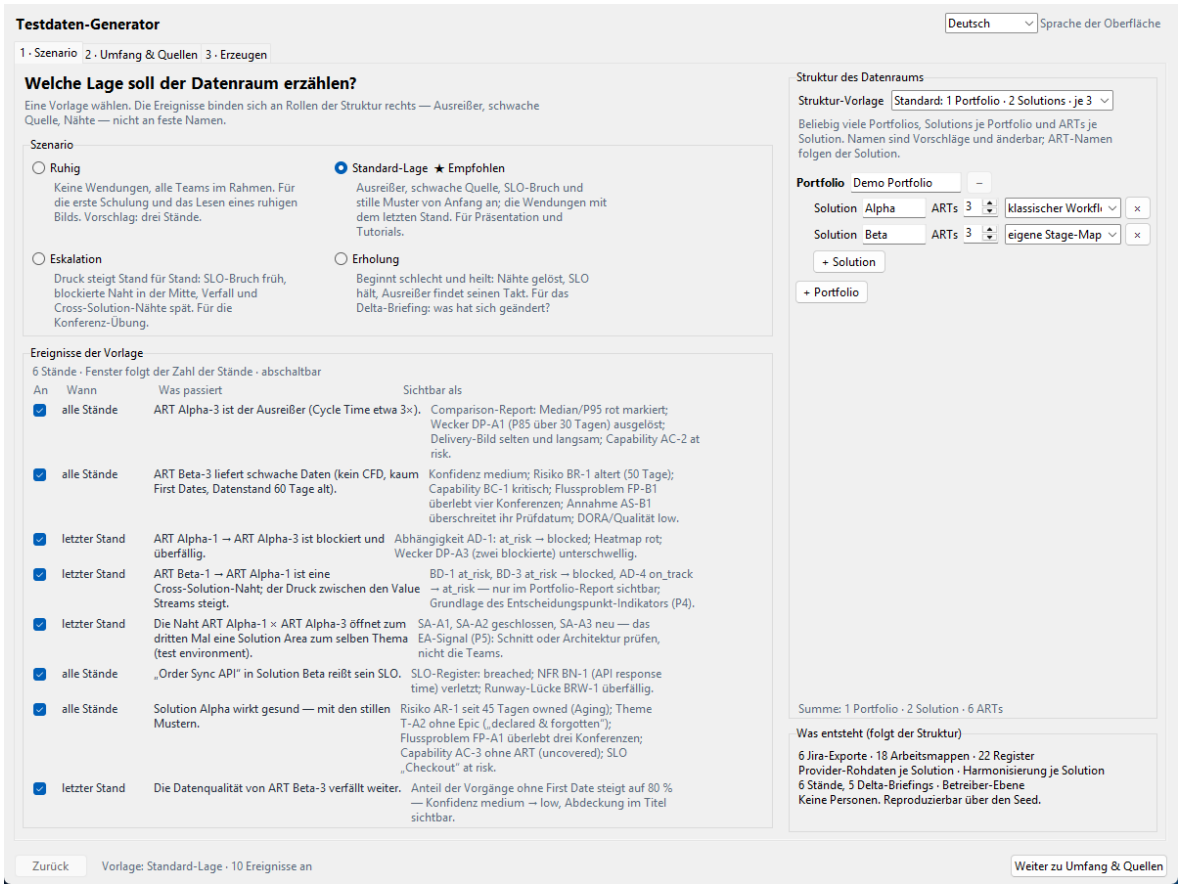


Abb. 1: Schritt 1 „Szenario" im Datenraum-Fenster — Vorlagen links, Ereignisse darunter, Struktur rechts

3.1 Schritt 1 — Szenario

Links wählst du eine **Vorlage** der Geschichte. Die Ereignisse binden sich an Rollen der Struktur rechts — Ausreißer, schwache Quelle, Nähte — nicht an feste Namen.

Vorlage	Was sie erzählt	Stände
Ruhig	Keine Wendungen, alle Teams im Rahmen. Für die erste Schulung und das Lesen eines ruhigen Bilds. Wer hier Alarm schlägt, hat das Bild überlesen.	3
Standard-Lage (empfohlen)	Ausreißer, schwache Quelle, SLO-Bruch und stille Muster von Anfang an; die Wendungen kommen mit dem letzten Stand. Für Präsentation und Tutorials.	6
Eskalation	Druck steigt Stand für Stand: SLO-Bruch früh, blockierte Naht in der Mitte, Verfall und Cross-Solution-Nähte spät. Für die Konferenz-Übung.	6

Vorlage	Was sie erzählt	Stände
Erholung	Beginnt schlecht und heilt: Nähte gelöst, SLO hält, Ausreißer findet seinen Takt. Für das Delta-Briefing: Was hat sich geändert?	6

Ereignisse der Vorlage. Die Tabelle darunter zeigt je Ereignis, wann es gilt (alle Stände, ab Stand n, letzter Stand), was passiert und als was es im Report sichtbar wird. Jedes Ereignis hat ein Häkchen; abgeschaltete Ereignisse fehlen im Datenraum. Ein Ereignis, dessen Voraussetzung aus ist, bleibt mit Hinweis aus. Das Fenster eines Ereignisses folgt der Zahl der Stände aus Schritt 2. Kapitel 5 erklärt die zehn Ereignisse.

Struktur des Datenraums (rechts). Eine Struktur-Vorlage — Klein (1 Portfolio · 1 Solution · 2 ARTs), Standard (1 · 2 · je 3), Groß (5 Portfolios · 3–4 Solutions · 1–6 ARTs) — oder eine eigene: Portfolios und Solutions hinzufügen oder entfernen, ARTs je Solution zählen, Namen ändern (ART-Namen folgen der Solution). Je Solution wählst du den klassischen Workflow oder eine eigene Stage-Map. Mindestens ein ART je Solution.

Was entsteht. Die Box unten rechts rechnet mit: Jira-Exporte, Arbeitsmappen, Register, Stände und Delta-Briefings für die gewählte Struktur (Standard: 6 Exporte, 18 Arbeitsmappen, 22 Register, 6 Stände, 5 Deltas). Ab 30 ARTs warnt sie: Die Erzeugung dauert dann spürbar.

3.2 Schritt 2 — Umfang & Quellen

Die Standardwerte reichen für Schulung und Präsentation.

Einstellung	Standard	Bedeutung
Stände	6	2 bis 12 Stände im Abstand von 14 Tagen; jeder Stand ist ein eingefrorener Snapshot, jedes Folgepaar ein fertiges Delta-Briefing. Die Vorlage „Ruhig“ schlägt 3 vor.
Umfang	M	S = nur Story-Anker, kurze Register; M = realistische Register; L = Stresstest mit vollen Registern (dichte Heatmap, volle Roadmap-Matrix). Die Anker und ihre Geschichten sind auf jeder Stufe identisch.
Realismus	60	Unordnung der Statuswege: Sprünge, Abbrüche, Parken, Schleifen, Wiedereröffnung. 0 = glatt, 100 = wie echt; 60 ist ein typischer ART.
Seed	42	Gleicher Seed, gleiche Struktur und gleiches Referenzdatum ergeben denselben Datenraum — ideal, wenn alle Teilnehmenden dasselbe sehen sollen.
Letzter Stand	heute	Datum des letzten Stands; die anderen liegen je 14 Tage davor. Die Qualitäts-Ampel des Portfolio-Reports stuft die Daten so als aktuell ein.
Sprache der Texte	Sprache der Oberfläche	Sprache von Geschichte, README und Delta-Briefings; folgt der Oberfläche.

Details für Analysten. Zykluszeiten, Flow-Muster und PI-Länge lassen sich je ART übersteuern — dieselben Regler wie beim einzelnen ART. In dieser Beta geht das über die Kommandozeile mit `--art-profiles DATEI.json` (Kapitel 6); die Tabelle in der Oberfläche folgt mit dem ART-Reiter.

Quellen, die mitgeliefert werden. Immer dabei: Jira-Exporte je ART (keine Personen), Prometheus-SLO-Antworten, GitHub-Deployments/PRs/Incidents für die erste Solution, GitLab-DORA-Tageswerte für die zweite, SonarQube-Qualitätsmaße und eine SLO-CSV für den manuellen Weg — Rohdaten im Format des jeweiligen Systems mit denselben Zahlen wie in den Registern, so dass `sources fetch` im Datenraum ohne Netz läuft.

KI-Entwürfe (Schalter, standardmäßig aus). Ein Mock-Lagevortrag, drei Antworten, ein Prompt-Bündel und eine Übungsantwort mit falschen Zahlen landen im Ordner ai_drafts/ — alle mit KI-Marker, keine freigegeben. Damit übst du die Freigabe und `llm check`, ohne ein Modell zu installieren.

Die Zeile **Schätzung** am Ende nennt Jira-Exporte, Register, Stände und Deltas für deine Einstellungen; die Fußzeile fasst Vorlage, Umfang, Stände, Realismus und Seed zusammen.

3.3 Schritt 3 — Erzeugen

Zielordner. Er wird angelegt. Ist er nicht leer, fragt der Generator nach — „Trotzdem erzeugen“ überschreibt Dateien gleichen Namens, „Anderen Ordner wählen“ geht zurück. „Durchsuchen …“ öffnet den Ordnerdialog; wo keiner verfügbar ist, trägst du den Pfad ein.

Fortschritt. „Datenraum erzeugen“ startet neun Schritte; jeder zeigt Laufen, Fertig oder Übersprungen mit einem Detail. Die Dauer liegt je nach Umfang bei Sekunden bis Minuten (Standard-Lage rund 10 Sekunden).

Schritt	Was entsteht
Struktur und Geschichte	structure.json und story.json: Rollen gebunden, Ereignisse festgelegt
Jira-Exporte und Arbeitsmappen	je ART ein Jira-Export unter raw/ und IssueTimes/Transitions/CFD unter solutions/…/arts/
Register	je Solution die Register (Risiken, Abhängigkeiten, NFR, SLO, DORA, Capabilities, Themen …)
Konfigurationen	portfolio.json, solution.json je Solution, pi_config.json
Provider-Rohdaten	sources/<solution>/ mit Replay-Dateien und sources-Configs
README	README.md mit Struktur, Geschichte, Ständen und weiteren Wegen
Stände-Reihe	snapshots/snapshot_<nn>_<datum>.json je Stand, stands.json, prev/now
Delta-Briefings	deltas/delta_<nn>-<nn+1>.md und .html je Folgepaar
KI-Entwürfe (Mock)	ai_drafts/ — nur mit Schalter, sonst „übersprungen“

Abbrechen hält nach dem laufenden Schritt an; der Ordner enthält, was bis dahin entstand. Ein Fehler erscheint mit Meldung in der Liste.

Ergebniskarte. Nach dem letzten Schritt erscheint „Datenraum fertig“ mit Dauer und Seed, vier Kennzahlen (Vorgänge und ARTs, Register und Solutions, Stände, Quellen-Arten), der Geschichte in drei Sätzen und Knöpfen: Portfolio-Report öffnen, Konferenzmappe, Jüngstes Delta, README, Ordner öffnen, KI-Entwürfe (wenn erzeugt). Die Reports werden beim ersten Klick gerendert und im Browser

geöffnet.

Delta vergleichen. Zwei Stände wählen (etwa 3 → 4) und „Delta-Briefing öffnen“ — alle Folgepaare liegen fertig unter deltas/.

Danach. „Neuen Datenraum mit anderer Vorlage …“ springt zu Schritt 1 zurück; die Zeile darunter zeigt den Befehl zum Fortschreiben (Kapitel 7).

4 Was im Ordner liegt

Der Zielordner ist ein **Portfolio-Datenraum**: Alle Pfade in den Configs sind relativ zur jeweiligen Config-Datei. Der Ordner lässt sich als Ganzes verschieben, kopieren oder zippen.

Pfad	Inhalt
`structure.json`	Struktur als Daten: Portfolios → Solutions → ARTs, Workflow je Solution
`story.json`	Geschichte als Daten: Vorlage, gebundene Rollen, Ereignisse mit Schalter und Stand-Fenster, Hinweise
`README.md`	Beschreibung von Struktur, Geschichte, Ständen, Quellen und weiteren Wegen — in der Sprache des Laufs
`portfolio.json`	Einstieg: Portfolio-Config über alle Solutions
`pi_config.json`	PI-Intervalle über den Datenzeitraum
`solutions/<name>/`	je Solution solution.json (eigene stage_map, wo gewählt), arts/ mit IssueTimes, Transitions und CFD je ART, registers/ mit den Registern unter Standardnamen
`raw/`	Jira-Exporte je ART im REST-API-Format — zugleich Demo-Futter für get_data
`sources/<name>/`	Provider-Rohdaten als Replay-Dateien (Prometheus, GitHub oder GitLab, SonarQube, CSV) plus sources-Configs
`snapshots/`	snapshot_<nn>_<datum>.json je Stand, Index stands.json, snapshot_prev/now.json = die letzten beiden
`deltas/`	delta_<nn>-<nn+1>.md und .html je Folgepaar — fertig gerendert
`ai_drafts/`	nur mit Schalter: Mock-Lagevortrag, Antworten, Prompt-Bündel, Übungsantwort — alle mit KI-Marker

Der Ordner ist direkt verwendbar — Portfolio-Report, Delta-Briefing eines beliebigen Paars, Konferenzmappe, `sources fetch` ohne Netz auf den Replay-Dateien (in sources/<solution>/) und die Export-Prüfung von get_data:

```
python -m portfolio portfolio.json
python -m portfolio --delta snapshots/snapshot_prev.json snapshots/snapshot_now.json
python -m portfolio portfolio.json --conference mappe.html
python -m sources fetch
python -m get_data check raw/<art>_jira.json
```

Bei mehreren Portfolios (Vorlage „Groß“ oder eigene Struktur) entsteht je Portfolio ein eigener Datenraum als Unterordner; eine README im Hauptordner verweist auf sie. Ein Lagebild über mehrere Portfolios hinweg ist heute kein Feature — die Daten dafür liegen bereits nebeneinander.

5 Die Geschichte verstehen

Jede Vorlage erzählt dieselbe Lage aus allen Registern heraus: Risiken, Abhängigkeiten, Annahmen, SLOs und Wecker zeigen auf dieselben Stellen. Die Ereignisse binden an **Rollen**, die aus der Struktur folgen — so bleibt die Geschichte auch bei eigener Struktur stimmig.

Die Rollen:
- **Erste Solution** („Gastgeber"): wirkt gesund, trägt die stillen Muster; ihr **letzter ART ist der Ausreißer**; zwischen ihren ARTs liegen die blockierte Abhängigkeit und die wiederkehrende Naht.
- **Zweite Solution:** ihr **letzter ART ist die schwache Datenquelle**; hier reißt die „Order Sync API" ihr SLO und erscheint das neue Sicherheitsrisiko.
- **Cross-Solution-Naht:** der erste ART der zweiten Solution hängt am ersten ART der ersten — nur im Portfolio-Report sichtbar.
- Weitere Solutions und Portfolios erhalten die seed-erzeugte Grundmenge ohne Story-Anker.

Was die Struktur nicht hergibt, entfällt mit **Hinweis** (in Schritt 1 und in story.json): Mit nur einer Solution gibt es keine schwache Quelle, keinen SLO-Bruch und keine Cross-Solution-Naht; hat die erste Solution nur einen ART, fehlen Ausreißer, blockierte Abhängigkeit und wiederkehrende Naht. Die zehn Ereignisse:

Nr.	Was passiert	Sichtbar als
E1	Der Ausreißer-ART hat etwa die dreifache Cycle Time.	Comparison-Report: Median/P95 rot; Wecker „P85 über 30 Tagen" ausgelöst; Delivery-Bild selten und langsam; eine Capability at risk.
E2	Die schwache Quelle liefert schwache Daten: kein CFD, kaum First Dates, Datenstand 60 Tage alt.	Konfidenz medium; ein Risiko altert (50 Tage); eine Capability kritisch; ein Flussproblem überlebt vier Konferenzen; eine Annahme überschreitet ihr Prüfdatum; DORA/Qualität low.
E3	Die Abhängigkeit zwischen zwei ARTs der ersten Solution ist blockiert und überfällig.	Abhängigkeit at_risk → blocked; Heatmap rot; Wecker „zwei blockierte" unterschwellig.
E4	Die Cross-Solution-Naht gerät unter Druck; die Value Streams ziehen aneinander.	Mehrere Abhängigkeiten rutschen von on_track über at_risk nach blocked — nur im Portfolio-Report sichtbar; Grundlage des Entscheidungspunkt-Indikators.
E5	Dieselbe Naht öffnet zum dritten Mal eine Solution Area zum selben Thema (Testumgebung).	Zwei Solution Areas geschlossen, eine neu — das EA-Signal: Schnitt oder Architektur prüfen, nicht die Teams.
E6	Die „Order Sync API" der zweiten Solution reißt ihr SLO.	SLO-Register: breached; NFR „API response time" verletzt; eine Runway-Lücke überfällig.
E7	Die erste Solution wirkt gesund — mit den stillen Mustern.	Ein Risiko seit 45 Tagen owned (Aging); ein Theme ohne Epic („declared & forgotten"); ein Flussproblem überlebt drei Konferenzen; eine Capability ohne ART; SLO „Checkout" at risk.

Nr.	Was passiert	Sichtbar als
E8	Die Datenqualität der schwachen Quelle verfällt weiter.	Anteil der Vorgänge ohne First Date steigt auf 80 % — Konfidenz medium → low, Abdeckung im Titel sichtbar.
E9	Neues Risiko in der zweiten Solution: Security-Review nicht terminiert.	Ein hohes, mitigiertes Risiko erscheint — im Delta-Briefing als neu.
E10	Roadmaps aktualisiert.	Ein Epic verliert sein Theme (Zombie-Initiative); ein Epic der zweiten Solution wird von P2 nach P1 gezogen.

Stand-Fenster. Jede Vorlage legt fest, in welchen Ständen ein Ereignis gilt. Standard-Lage: Ausreißer, schwache Quelle, SLO-Bruch und stille Muster in allen Ständen, die Wendungen (Blockade, Nähte, Verfall, neues Risiko, Roadmaps) nur im letzten. Eskalation staffelt sie über die Reihe, Erholung kehrt sie um: Stand 1 zeigt alles, dann heilt eines nach dem anderen. Die README des Datenraums listet je Ereignis sein Fenster; davor und danach zeigt die Reihe die Lage ohne es.

6 Der Datenraum auf der Kommandozeile

Alles aus den drei Schritten gibt es auch als Option von `python -m testdata_generator --scenario portfolio`. Vor dem Bau nennt die Kommandozeile Struktur, Schalter und Schätzung; danach Ordner, Stände und Dauer.

Option	Standard	Bedeutung
--scenario portfolio	—	Datenraum statt einzelnem ART erzeugen
--output ORDNER	(Pflicht)	Zielordner (wird angelegt)
--structure small|standard|large|FILE.json	standard	Struktur-Vorlage oder eigene structure.json (Portfolios → Solutions → ARTs, Workflow je Solution)
--arts N[,N...]	—	Ein Portfolio aus ART-Zahlen je Solution, z. B. 3,4s,1 — drei Solutions mit 3, 4 und 1 ARTs; ein angehängtes s gibt der Solution eine eigene Stage-Map. Nicht mit --structure kombinierbar.
--story calm|standard|escalation|recovery	standard	Vorlage der Geschichte (Ruhig, Standard-Lage, Eskalation, Erholung)
--event ID=on|off	—	Ein Ereignis an- oder abschalten, z. B. --event E3=off --event E9=on; wiederholbar. Ein Ereignis, dessen Voraussetzung aus ist, bleibt mit Hinweis aus.
--stands N	6 (Ruhig 3)	Zahl der Stände (2–12), 14 Tage Abstand, der letzte auf dem Referenzdatum; jedes Folgepaar bekommt ein Delta-Briefing unter deltas/
--scale s|m|l	m	Umfang der Register je Solution: s = nur Anker, m = realistische Demo, l = Stresstest
--seed N	(zufällig)	Seed für reproduzierbare Ausgabe; die Oberfläche nimmt 42
--art-profiles FILE.json	—	Regler je ART als JSON, z. B. {"Alpha-1": {"mean_cycle_days": 30, "pattern": "cluster", "pattern_strength": 80}}. Felder: issue_count, mean_cycle_days, std_cycle_days, completion_rate, todo_rate, backflow_prob, pattern, pattern_strength, pi_duration_weeks, realism. Overrides gelten in allen Ständen; alles andere behält seinen Story-Wert.
--ai-drafts	aus	Mock-KI-Entwürfe mit Marker nach ai_drafts/ schreiben
--lang de|en|ro|pt|fr	Suite-Sprache, sonst en	Sprache von Oberfläche, Konsole und erzeugten Texten
--continue ORDNER --to-date DATE	—	Einen bestehenden Datenraum bis zum Datum fortschreiben (Kapitel 7)

Option	Standard	Bedeutung
--web / --port N / --no-browser	—	Browser-Oberfläche starten: lokaler Server auf 127.0.0.1 (Port 0 = zufällig frei), Zugangsschlüssel im Link; --no-browser druckt nur den Link. Ende mit Strg+C.
--dataspace	—	Datenraum-Fenster starten — dieselben drei Schritte ohne Server

Beispiele

```
# Standard-Lage, wie in Tutorial und Handbüchern
python -m testdata_generator --scenario portfolio --output demo/ --seed 42

# Eskalation über acht Stände, Stresstest, deutsche Texte
python -m testdata_generator --scenario portfolio --output demo-esc/ \
    --story escalation --stands 8 --scale l --lang de

# Eigene Struktur, ein Ereignis aus, KI-Übung an
python -m testdata_generator --scenario portfolio --output demo-3/ \
    --arts 3,4s,1 --event E3=off --ai-drafts

# Fünf Portfolios, 49 ARTs — dauert spürbar
python -m testdata_generator --scenario portfolio --structure large \
    --output demo-large/
```

Ohne --lang und ohne Launcher-Einstellung sind Konsole und erzeugte Texte englisch. Wer Deutsch will, gibt --lang de mit oder stellt den Launcher um.

7 Fortschreiben

Ein Datenraum lässt sich zu einem späteren Datum fortschreiben — wie ein zweiter Abzug aus Jira: Die Vergangenheit bleibt, offene Vorgänge laufen weiter, neue kommen dazu.

```
python -m testdata_generator --scenario portfolio \
    --continue demo/ --to-date 2027-01-15
```

Der Generator schreibt die Jira-Exporte, IssueTimes/Transitions/CFD, Configs und die PI-Config im Ordner fort; der bisherige snapshot_now wird snapshot_prev, der neue Stand snapshot_now, und die README erhält einen Nachtrag mit altem und neuem Referenzdatum und der Zahl neuer Vorgänge. Die Register bleiben unverändert. Die Regler kommen aus dem Generator-Protokoll der Dateien; die Sprache folgt `--lang`.

> *Die Ergebniskarte zeigt den passenden Befehl unter „Danach“. Für einen einzelnen Jira-Export gibt es --continue OLD.json (Kapitel 8).*

8 Ein einzelner ART (klassischer Generator)

Der klassische Weg erzeugt **eine** Jira-Export-Datei im REST-API-Format aus einer Workflow-Datei: Vorgänge durchlaufen die Stages, verbleiben unterschiedlich lange, fallen gelegentlich zurück und schließen mit einstellbarer Quote ab. Die Datei geht direkt in transform_data und build_reports. Die Regler für Cycle Time und Flow-Muster sind dieselben, die der Datenraum je ART über --art-profiles übersteuert.

8.1 Das Fenster

TestdataGenerator.bat (bzw. `python -m testdata_generator` ohne Option) öffnet das Fenster mit Eingabefeldern für alle Parameter. Pflichtfeld ist nur die Workflow-Datei; alles andere hat Standardwerte. Der Bereich „Demo-Portfolio“ unten erzeugt die Standard-Lage per Knopf — der Datenraum aus Kapitel 3 kann mehr.

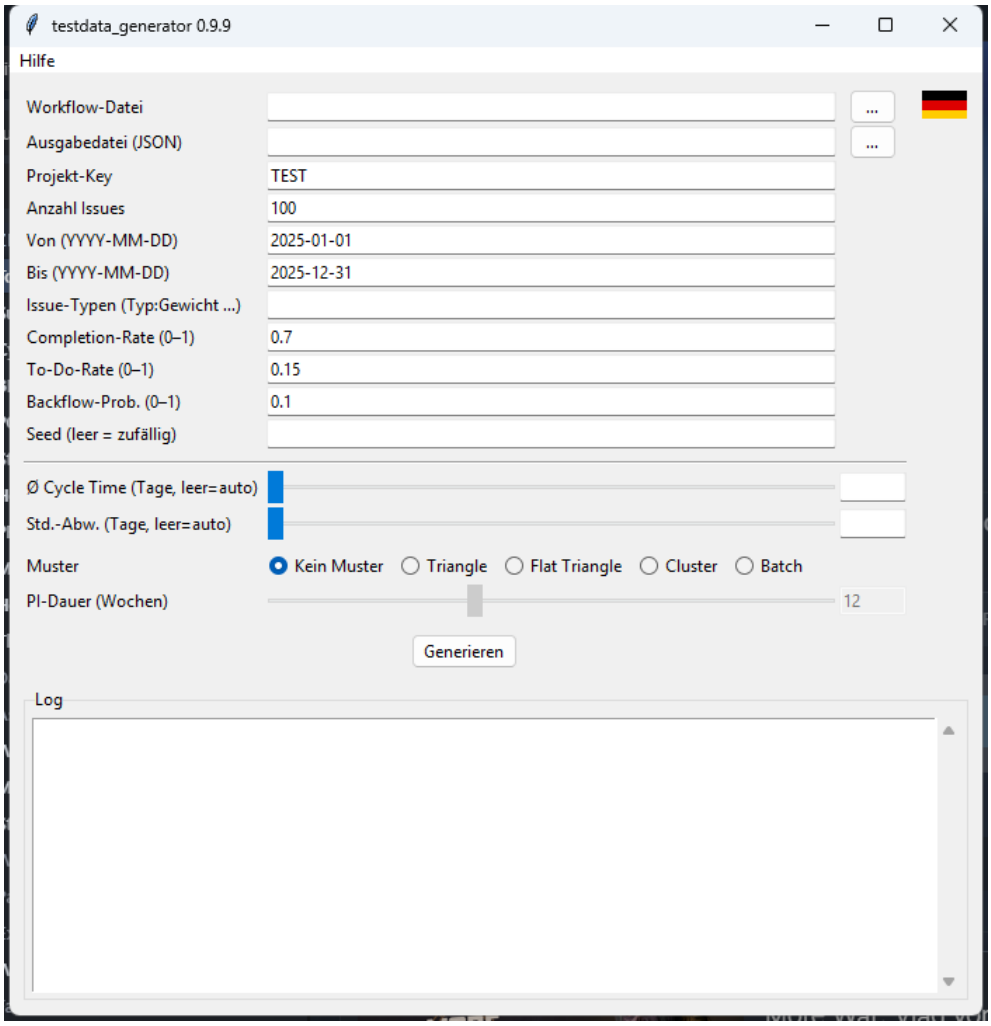


Abb. 2: Fenster des klassischen Generators für einen einzelnen ART

8.2 Die Workflow-Datei

Die Workflow-Datei definiert die Stages deines Jira-Boards und ist das einzige Pflichtfeld. Sie hat dasselbe Format wie in transform_data — eine Datei genügt für beide Module. Jede Zeile beschreibt eine Stage; Aliases (alternative Status-Namen im Jira-Export) werden durch Doppelpunkte abgetrennt. Zwei Sonderzeilen markieren Anfang und Ende der aktiven Bearbeitung:

```
KanonischerName:Alias1:Alias2
<First>ErsteAktiveStage
<Closed>AbgeschlosseneStage
```

Zeile	Bedeutung
Name:Alias1:Alias2	Stage mit einem oder mehreren alternativen Jira-Status-Namen
Name (ohne :)	Stage ohne Aliases
<First>Name	Erste aktive Stage — hier beginnt die Cycle Time
<Closed>Name	Abgeschlossene Stage — markiert einen Vorgang als fertig

Das mitgelieferte Beispiel `workflow_ART_A.txt` bildet einen typischen SAFe-ART-Workflow ab:

```
Funnel:New:Open:To Do
Analysis:In Analysis:Estimated
Program Backlog:Ready for Dev:Backlog
Implementation:In Implementation:In Review:In Progress
Blocker
Validating on Staging:QA
Deploying to Production:Ready for Development
Releasing:Completed
Done:Canceled
<First>Analysis
<Closed>Releasing
```

„Funnel“ hat vier Aliases — alle vier Jira-Status werden auf Funnel normiert. Analysis ist die erste aktive Stage (Cycle Time beginnt hier). Releasing gilt als abgeschlossen; Done (nach Releasing) nimmt auch Canceled-Vorgänge auf.

8.3 Parameter-Referenz

Alle Parameter gibt es im Fenster und auf der Kommandozeile.

Parameter	Standard	Beschreibung
--workflow FILE	(Pflicht)	Workflow-Definitionsdatei (.txt)
--output FILE.json	_generated.json	Pfad der Ausgabedatei
--project KEY	TEST	Jira-Projekt-Key der erzeugten Vorgänge
--issues N	100	Zahl der Vorgänge
--from-date YYYY-MM-DD	2025-01-01	Frühestes Erstellungsdatum
--to-date YYYY-MM-DD	2025-12-31	Spätestes Übergangsdatum
--issue-types TYPE:W ...	Feature:0.6 Bug:0.3 Enabler:0.1	Vorgangstypen mit Gewichtung (Summe muss nicht 1 ergeben)
--completion-rate FLOAT	0.7	Anteil der Vorgänge, die die Closed-Stage erreichen (0–1)
--todo-rate FLOAT	0.15	Anteil offener Vorgänge, die in frühen Stages verbleiben (0–1)
--backflow-prob FLOAT	0.1	Wahrscheinlichkeit eines Rückschritts je Übergang (0–1)
--realism 0-100	0	Unordnung der Statuswege: Sprünge, Abbrüche, Parken, Schleifen, Wiedereröffnung; etwa 70 wie ein echter ART
--seed INT	(zufällig)	Seed für reproduzierbare Ausgabe
--mean-cycle-days FLOAT	—	Mittlere Cycle Time in Tagen (lognormal); schaltet die Cycle-Time-Steuerung ein
--std-cycle-days FLOAT	30 % des Mittels	Standardabweichung der Cycle Time in Tagen
--pattern NAME	none	Flow-Muster: none / triangle / flat_triangle / cluster / batch (siehe 8.4)
--pi-duration-weeks INT	12	PI-Länge in Wochen (für cluster und batch)
--pattern-strength 0-100	50	Stärke des Musters (0 = subtil, 50 = Standard, 100 = stark)
--continue OLD.json	—	Früheren Abzug bis --to-date fortschreiben: Vergangenheit bleibt Byte für Byte, offene Vorgänge laufen weiter, neue kommen dazu; braucht --workflow
--learn-from FILE.xlsx	—	Übergangsprofil aus einer echten Transitions-Datei lernen (nur Zählungen und Stunden); --transition-profile erzeugt damit

8.4 Flow-Muster

Der Generator simuliert typische Flow-Antipattern aus der Literatur von Vacanti und Singh. Dazu wird --mean-cycle-days mit --pattern kombiniert; im Fenster gibt es Auswahlknöpfe und Schieberegler.

Muster	Scatterplot	Beschreibung
none	Gleichmäßige Punktwolke	Zufällige Cycle Time ohne Trend — Standardverhalten
triangle	Dreieck, rechter Winkel unten rechts	Cycle Time steigt linear über die Zeit (etwa 1–4× bei Standardstärke). Ein Prozess, der zunehmend langsamer wird.
flat_triangle	Dreieck, Anstieg flacht ab	Wie triangle, der Anstieg verflacht am Ende. Eine Verschlechterung, die sich stabilisiert.
cluster	Senkrechte Häufungen am PI-Ende	Cycle Time bleibt stabil, aber die meisten Vorgänge schließen in den letzten zwei Wochen vor PI-Ende. Deadline-getriebenes Verhalten.
batch	Säulen variabler Höhe am PI-Ende	Wie cluster, aber mit stark unterschiedlicher Cycle Time (0,1× bis 3× Mittel). Kurze und sehr lange Aufgaben werden gemeinsam ans PI-Ende geschoben.

--pattern erfordert --mean-cycle-days; ohne wird das Muster ignoriert. --std-cycle-days steuert die Streuung, --pi-duration-weeks die PI-Länge für cluster und batch.

8.5 ART_A — vollständiges Beispiel

Vom Generator bis zum fertigen Report am fiktiven Projekt ART_A:

Schritt 1: Testdaten erzeugen

```
python -m testdata_generator \
    --workflow testdata_generator/workflow_ART_A.txt \
    --project ART_A --issues 200 \
    --from-date 2025-01-01 --to-date 2025-12-31 \
    --issue-types Feature:0.6 Bug:0.2 Enabler:0.2 \
    --completion-rate 0.75 --backflow-prob 0.08 \
    --seed 42 --output ART_A_generated.json
```

Schritt 2: Mit transform_data verarbeiten

```
python -m transform_data ART_A_generated.json \
    --workflow testdata_generator/workflow_ART_A.txt
```

Schritt 3: Report mit build_reports erstellen

```
python -m build_reports
```

Was zu erwarten ist (Seed 42, 200 Vorgänge):
- Keys ART_A-0001 bis ART_A-0200
- Typen etwa 120 Feature, 40 Bug, 40 Enabler
- Etwa 150 Vorgänge in Releasing oder Done
- Erstellungsdaten über das Jahr 2025 verteilt
- Gelegentliche Rückschritte (backflow-prob 0.08 = etwa 8 %)

9 Häufige Fragen

Fenster oder Browser — was soll ich nehmen?

Das Fenster. Es erzeugt denselben Datenraum und öffnet keinen Netzwerkport. Die Browser-Seite nur auf einem privaten Rechner; auf Firmenrechnern blockiert und meldet die Sicherheitssoftware lokale Server.

Warum ist mein Datenraum englisch, obwohl ich Deutsch spreche?

Ohne Sprachangabe nimmt der Generator die Suite-Sprache aus dem Launcher, sonst Englisch. Stell den Launcher auf Deutsch, schalte oben rechts um oder gib --lang de mit.

Der Zielordner ist nicht leer — was passiert?

Der Generator fragt nach. „Trotzdem erzeugen“ überschreibt Dateien gleichen Namens und lässt alles andere stehen; „Anderen Ordner wählen“ bricht ab.

Die Erzeugung dauert lange.

Ab 30 ARTs oder bei Umfang L mit mehreren Portfolios warnt die Box „Was entsteht“. Für Schulungen reichen Standard-Struktur und Umfang M (rund 10 Sekunden).

Testdaten oder echte Daten?

Für Demos, Schulungen und Installationstests sind Testdaten die bessere Wahl — ohne Datenschutzfrage. Für echte Flow-Analysen und Entscheidungen braucht es echte Daten; den manuellen Export beschreibt das Handbuch Get Data.

Kann ich einzelne ARTs anders einstellen?

Ja, über --art-profiles DATEI.json auf der Kommandozeile (Kapitel 6). Der Reiter dafür in der Oberfläche folgt in einem späteren Beta-Stand.

10 Glossar

Begriff	Erklärung
ART	Agile Release Train — mehrere Teams, die in SAFe gemeinsam liefern
Datenraum	Ein Ordner mit allem, was ein Portfolio-Lagebild braucht: Configs, Exporte, Register, Stände, Briefings — in sich geschlossen und verschiebbar
Delta-Briefing	„Was hat sich geändert?“ zwischen zwei Ständen — Durchsatz, Konfidenz, Nähte, neue Risiken
Konferenzmappe	Die Mappe für die Value Stream Conference: Flussprobleme, die Konferenzen überleben, mit dem Lagebild dazu
Naht	Eine Abhängigkeit zwischen ARTs oder Solutions, an der sich Druck zeigt — innerhalb einer Solution oder zwischen den Value Streams
Register	Eine Liste je Solution: Risiken, Abhängigkeiten, Annahmen, NFR, Runway, SLO, DORA, Capabilities, Themen, Flussprobleme …
Stand	Ein eingefrorener Snapshot des Lagebilds zu einem Datum; die Stände liegen 14 Tage auseinander
Seed	Startwert des Zufallsgenerators — gleicher Seed, gleiche Ausgabe
Solution	Eine Gruppe von ARTs, die gemeinsam eine Lösung liefert; im Datenraum die Ebene mit eigenen Registern
Stage-Map	Eigene Zusammenfassung der Workflow-Stages einer Solution (etwa Vorlauf/Umsetzung/Fertig) statt des Default-Pfads
Value Stream	Der Fluss vom Bedarf bis zum Wert; im Datenraum durch die Solutions vertreten
Workflow	Geordnete Abfolge von Stages, die ein Vorgang durchläuft; als Textdatei beschrieben
Realismus	Grad der Unordnung in den Statuswegen: Sprünge, Abbrüche, Parken, Schleifen, Wiedereröffnung
CFD	Cumulative Flow Diagram — Bestand je Stage über die Zeit, kumuliert